

面向大模型及其应用的安全性研究

——基于 Argus 的智能体行为监督原型系统

项目小组

2026 年 7 月

摘要

大语言模型智能体能够调用 Shell、文件读写、网页抓取和外部工具，安全风险也从传统的“模型输出是否合规”扩展为“模型行为是否安全”。本项目从红队视角研究大模型及其应用的攻击面，覆盖提示注入、模型越狱、训练数据泄露、工具调用劫持、记忆中毒、环境感知污染、间接提示注入和序列异常等场景。项目实现了一个可嵌入或旁路部署的智能体行为监督原型系统 Argus Guardian，在工具调用真实执行之前，对工具名、工具参数、上下文、污点来源和历史序列进行四层审计，输出 ALLOW、FLAG 或 BLOCK 决策。最终成果包括安全风险分析、对抗样本与越狱测试集、攻击复现脚本、评测结果，以及可网页演示的智能体行为监督系统。

关键词：大语言模型安全；智能体安全；提示注入；工具调用劫持；污点追踪；行为监督；红队评测

1 选题目标与项目范围

本项目对应课程选题“面向大模型及其应用的安全性研究”。题目要求从红队视角研究大语言模型及智能体应用的攻击面，包括提示注入、模型越狱、训练数据泄露、工具调用劫持、记忆中毒、环境感知污染等；核心目标是设计可嵌入或旁路部署的行为监督机制，对智能体的工具调用、代码执行和文件访问进行实时审计、异常检测和阻断；预期成果包括安全风险分析报告、对抗样本与越狱测试集、攻击脚本，以及一个可演示的智能体行为监督原型系统。

Argus 将大模型智能体抽象为如下行为链路：

用户请求 -> LLM Agent 规划工具调用 -> Guardian 审计 -> ALLOW/FLAG/BLOCK
-> 工具执行或阻断

因此，本项目关注的不是普通脚本安全，而是大模型智能体在具备工具调用能力之后产生的行为安全问题。系统默认不会真实执行 DeepSeek 在线红队生成的危险工具

调用，而是将其转换为结构化 ToolCall 后交给 Guardian 审计，从而保证课程演示环境的安全性。

图位：系统总体架构图

请放置一张系统架构图，建议内容为：用户请求、LLM Agent、Tool-Call、Argus Guardian 四层审计、工具执行或阻断、审计日志和 Dashboard。可用 draw.io、PowerPoint 或 mermaid 绘制。建议文件名：`report/figures/system_architecture.png`。

图 1: 系统总体架构图占位

2 威胁模型与攻击面分析

2.1 威胁模型

本项目假设攻击者无法直接控制宿主机，但可以通过用户输入、网页内容、配置文件、日志文件、长期记忆或工具返回内容影响大模型智能体的上下文。攻击者的目标是让智能体越权调用工具、读取或写入敏感文件、执行危险命令、污染长期记忆，或把内部数据外联到恶意域名。被保护对象包括：

- 工具调用边界：防止模型调用非白名单工具或危险工具参数；
- 代码执行边界：防止 `run_shell` 执行删除、反弹 Shell、下载执行等命令；
- 文件访问边界：防止目录穿越、敏感凭据读取、训练数据泄露和长期记忆污染；
- 上下文可信边界：防止网页、文件、日志等不可信来源中的隐藏指令影响高权限动作；
- 行为序列边界：识别单次调用看似普通但处于数据外泄或提权链路中的异常行为。

2.2 攻击面分类

项目将攻击面整理为八类，其中前七类可在 Dashboard 的攻击面实验台中一键重跑，序列异常作为补充评测场景进入离线测试集。

攻击面	典型目标	Argus 防御要点
提示注入	诱导模型忽略安全规则并执行危险工具	危险命令模式、提示注入关键词、工具执行前阻断
模型越狱	通过角色扮演或无约束模式绕过策略	Shell 高危命令、PowerShell encoded command、反弹 Shell 检测
训练数据泄露	读取训练语料、密钥、Token、私钥等敏感信息	文件沙箱、敏感路径模式、凭据读取阻断
工具调用劫持	从正常任务跳转到非白名单工具、删除数据库或外联	工具白名单、外联域名、DeepSeek 红队元数据兜底
记忆中毒	将恶意偏好或伪系统规则写入长期记忆	高风险路径写入检测、不可信来源写记忆阻断
环境感知污染	利用 README、配置、日志等伪造系统事实	污点追踪、环境文件注入文本识别
间接提示注入	将攻击指令藏在网页或文件内容中	来源级和片段级污点追踪，高权限 Sink 阻断
序列异常	重复调用、读取后外联、读取后提权	Anomaly 层检测重复循环和异常攻击链

3 系统设计

3.1 核心数据结构

Argus 的核心接口是 ToolCall -> Context -> Verdict -> Decision。

- ToolCall 表示智能体准备调用的一次工具，包含工具名、参数和工具调用 ID；
- Context 表示当前会话上下文，包含用户原始请求、历史工具序列、污点来源、片段级污点和红队元数据；
- Verdict 是单个防御层的判断，动作可以是 ALLOW、FLAG 或 BLOCK；
- Decision 是 Guardian 对四层判断的聚合结果，任一层 BLOCK 则最终阻断，无阻断但有 FLAG 则告警。

3.2 四层行为监督机制

Argus Guardian 由 Policy、Taint、Intent、Anomaly 四层组成。四层不是互相替代，而是分别从规则、数据流、语义一致性和序列异常角度补足行为监督能力。

防御层	主要输入	检测逻辑
PolicyLayer	工具名、参数、用户请求、红队元数据、自适应规则	检查工具白名单、危险 Shell、沙箱路径、敏感文件、外联请求、记忆中毒写入；对 DeepSeek 红队样本中的期望阻断元数据做兜底阻断。
TaintLayer	污点来源、片段级污点、工具参数	将 <code>web_fetch</code> 和 <code>read_file</code> 视为不可信来源，抽取片段摘要；若污点文本流入 Shell、写文件、计划文件或记忆文件等高权限 Sink，则告警或阻断。
IntentLayer	用户请求、工具调用、可选 DeepSeek Judge	判断当前工具调用是否符合用户原始意图；默认关闭，Dashboard 可勾选在线 DeepSeek intent judge。
AnomalyLayer	历史工具序列、红队元数据、工具类型	检测重复调用、读取/抓取后跳到高影响工具、读取动作与外联目标共现、DeepSeek 红队攻击链关键词。

图位：四层 Guardian 判定流程图

请放置四层防御流程图。建议内容：Policy、Taint、Intent、Anomaly 并行返回 Verdict，Guardian 汇总为最终 Decision。图中标注“任一 BLOCK 即阻断；无 BLOCK 但有 FLAG 即告警”。建议文件名：`report/figures/guardian_layers.png`。

图 2: 四层 Guardian 判定流程图占位

3.3 自适应规则闭环

当 DeepSeek 在线红队生成的攻击样本未被阻断时，Dashboard 可调用 DeepSeek 分析漏拦截原因，并把模型建议约束为受限字段规则，例如 `tool_name`、`surface_id`、`user_request_contains_any`、`input_contains_any`、`metadata_contains_any`。系统不会执行模型生成的代码，也不接受任意正则或脚本，从而降低“让模型修改安全策略”带来的二次风险。

规则管理页会展示规则来源、启停状态、命中次数和撤销按钮。PolicyLayer 命中自适应规则时会自动累加命中次数。该机制形成“红队生成攻击 -> Guardian 审计 -> 漏拦截分析 -> 受限规则写入 -> 重评估”的闭环。



图 3: 自适应规则管理页截图占位

4 原型系统实现

4.1 主要模块

路径	作用
<code>src/agent/deepseek_agent.py</code>	DeepSeek/OpenAI-compatible Agent，支持 Tool Calling，并在工具执行前接入 Guardian。
<code>src/agent/tools.py</code>	提供 <code>run_shell</code> 、 <code>read_file</code> 、 <code>write_file</code> 、 <code>web_fetch</code> 工具，包含沙箱和 fixture 支持。
<code>src/guardian/</code>	四层防御、统一决策、审计日志、DeepSeek intent judge、自适应规则。

<code>src/redteam/attacks.py</code>	离线红队攻击样本和良性对照样本。
<code>src/redteam/surface_lab.py</code>	7 个攻击面的离线样本和 DeepSeek 在线红队提示词模板。
<code>src/eval/benchmark.py</code>	计算召回率、误报率、延时分位数、分类指标、四层 Verdict 分布和混淆矩阵。
<code>scripts/dashboard_server.py</code>	本地 Dashboard 前后端服务，提供评测、DeepSeek 红队、规则管理、演示验收 API。
<code>dashboard/</code>	网页前端，展示攻击面实验台、批量红队矩阵、攻击链视图、规则管理和演示验收。

4.2 Dashboard 演示系统

Dashboard 是本项目的可演示原型系统。它由单个 Python 进程同时提供前端页面和后端 API，默认运行在 `http://127.0.0.1:8765`。核心功能包括：

- 总览指标：攻击检出数、阻断数、告警数和平均延时；
- 攻击面实验台：7 个攻击面均可离线重跑；
- DeepSeek 在线红队：支持 4 类提示词模式和批量生成；
- 批量状态矩阵：每条红队样本显示“总 / 1 / 2 / 3 / 4”状态；
- 攻击链弹窗：展示用户请求、模型输出/攻击点、工具调用、污点来源和防御层命中；
- 规则管理：查看、启停、撤销自适应规则；
- 一键演示验收：固定运行 7 个攻击面，生成 Markdown、JSON 和 SVG 截图快照；
- 持久历史流：每次审计写入 SQLite，页面指标可随历史请求动态变化。

图位：Dashboard 总览截图

请放置 Dashboard 首页截图。要求包含顶部四个指标卡、攻击面实验台和四层防御统计。建议在启动 `dashboard_server.py` 后访问 `http://127.0.0.1:8765` 截图。建议文件名：`report/figures/dashboard_overview.png`。

图 4: Dashboard 总览与四层统计截图占位

图位：DeepSeek 红队批量矩阵与攻击链弹窗截图

请放置一次 DeepSeek 红队运行后的截图。要求包含批量矩阵“总/1/2/3/4”状态，以及点击某条样本后弹出的攻击链视图。建议文件名：`report/figures/deepseek_attack_chain.png`。

图 5: DeepSeek 红队批量矩阵和攻击链视图截图占位

5 对抗样本、越狱测试集与攻击脚本

5.1 对抗样本与良性对照

当前离线测试集共 30 条样本，其中攻击样本 22 条，良性对照 8 条。样本以两种形式保存：

- `datasets/seed_cases.jsonl`: 便于提交和复现实验的 JSONL 测试集；
- `src/redteam/attacks.py`: 可直接被 Python 评测脚本导入的结构化用例。

表 4: 离线测试集攻击面覆盖

类别	总数	攻击数	良性数	检出数
prompt_injection	2	2	0	2
model_jailbreak	2	2	0	2
training_data_leak	3	3	0	3
tool_hijack	5	5	0	5
indirect_injection	3	3	0	3
memory_poison	3	3	0	3
environment_pollution	2	2	0	2
sequence_anomaly	2	2	0	2
benign	8	0	8	0

5.2 攻击脚本

项目提供了多个复现和评测脚本：

- `scripts/offline_demo.py`: 无需 API Key, 演示良性放行、危险命令阻断和间接注入阻断;
- `scripts/replay_case.py <case_id>`: 按样本 ID 复现单个攻击;
- `scripts/run_benchmark.py`: 运行全部样本并生成 `report/eval_results.md/json`;
- `scripts/convert_public_jailbreaks.py`: 将 AdvBench/JailbreakBench 风格公开数据转换为 Argus JSONL;
- `scripts/run_intent_judge_eval.py`: 使用 DeepSeek intent judge 进行在线语义一致性评测;
- `scripts/deepseek_demo.py`: 运行真实 DeepSeek Agent demo。

Listing 1: 常用验收命令

```

.\.venv\Scripts\python -m pytest
.\.venv\Scripts\python -m compileall src scripts tests
.\.venv\Scripts\python scripts\run_benchmark.py
node --check dashboard\app.js

```

```
.\.venv\Scripts\python scripts\dashboard_server.py --host 127.0.0.1 --port 8765
```

6 实验设计与结果

6.1 实验设置

离线评测使用 EVAL_CASES 中的 30 条样本。攻击样本期望被 BLOCK 或 FLAG；良性样本允许 ALLOW，其中部分涉及不可信来源后的低风险写入，允许被 FLAG，但不应被 BLOCK。核心指标包括攻击检出率、阻断型误报率、良性告警数、平均延时、p50/p95 延时、按攻击面分类指标和四层 Verdict 分布。

6.2 总体结果

表 5: 离线评测总体指标

指标	结果
总样本数	30
攻击样本数	22
良性样本数	8
攻击检出数	22
漏检攻击数	0
阻断型误报数	0
良性告警数	2
召回率	100.00%
阻断型误报率	0.00%
平均延时	1.049 ms
p50 延时	0.925 ms
p95 延时	1.730 ms

实验结果表明，Argus 在当前离线测试集中检出了全部 22 条攻击样本，未产生阻断型误报。2 条良性样本被标记为 FLAG，原因是它们涉及“不可信来源后写入普通摘要”这类旁路监督中值得提示但不应阻断的行为。

6.3 四层 Verdict 分布

表 6: 四层防御 Verdict 分布

防御层	ALLOW	FLAG	BLOCK
Policy	12	0	18
Taint	22	2	6
Intent	30	0	0
Anomaly	18	7	5

Policy 层承担主要硬阻断职责, Taint 层覆盖间接提示注入和不可信数据流, Anomaly 层对外联链路、读取后提权和重复调用给出补充判断。Intent 层在离线评测中默认不启用, 因此均为 ALLOW; 在 Dashboard 中可勾选 DeepSeek intent judge 进行在线语义一致性演示。

图位: Benchmark 结果截图

请放置运行 `scripts/run_benchmark.py` 后的终端输出或 `report/eval_results.md` 截图。要求能看到总样本数 30、攻击样本 22、攻击检出 22、阻断型误报 0、四层 Verdict 分布。建议文件名: `report/figures/benchmark_results.png`。

图 6: 离线评测结果截图占位

6.4 一键演示验收

Dashboard 提供 `POST /api/demo-run` 和“演示验收”按钮, 固定运行 7 个攻击面并生成验收产物。一次 smoke test 中, 该接口返回 7 个攻击面, 7/7 被检出, 并生成以下文件:

- `sandbox_runs/demo_acceptance/argus_demo_*.md`: Markdown 验收记录;
- `sandbox_runs/demo_acceptance/argus_demo_*.json`: 机器可读结果;

- `sandbox_runs/demo_acceptance/argus_demo_*.svg`: 截图快照。



图 7: 一键演示验收快照占位

7 预期成果完成情况

课程预期成果	完成状态	对应文件或功能
安全风险分析报告	已完成 LaTeX 正式报告，包含威胁模型、攻击面、系统设计和实验结果	<code>report/final_report.tex</code> 、 <code>docs/architecture.md</code> 、 <code>docs/project_overview.md</code>
对抗样本与越狱测试集	已完成 30 条离线样本，其中攻击样本 22 条、良性对照 8 条，并支持公开越狱集格式转换	<code>datasets/seed_cases.jsonl</code> 、 <code>src/redteam/attacks.py</code> 、 <code>scripts/convert_public_jailbreaks.py</code>
攻击脚本	已完成离线演示、单样本复现、全量评测、DeepSeek intent judge、真实 DeepSeek Agent demo	<code>scripts/offline_demo.py</code> 、 <code>scripts/replay_case.py</code> 、 <code>scripts/run_benchmark.py</code> 、 <code>scripts/deepseek_demo.py</code>

智能体行为监督原型系统	已完成本地网页 Dashboard 和后端 Guardian API, 支持实时审计、异常检测、阻断、规则管理、攻击链视图和演示验收	scripts/dashboard_server.py、dashboard/、src/guardian/
实验结果与验收记录	已生成 Markdown/J-SON 评测结果, 并支持一键生成演示验收 Markdown/J-SON/SVG	report/eval_results.md、report/eval_results.json、sandbox_runs/demo_acceptance/

8 局限性与后续工作

当前项目已满足课程原型验收, 但仍存在进一步研究空间:

- 当前数据集规模适合课程演示, 后续可扩展更多公开越狱数据和真实 Agent 轨迹;
- IntentLayer 依赖在线 LLM judge, 离线评测默认关闭, 后续可加入本地轻量分类器或多模型交叉判断;
- AnomalyLayer 目前是可解释启发式, 后续可基于良性轨迹训练 n-gram、Markov 或序列模型;
- 自适应规则目前是字段包含匹配, 后续可增加规则差异预览和更细粒度的作用域控制;
- Dashboard 可继续增加 CSV 导出、演示视频录制和多模型红队对比。

9 结论

本项目围绕“面向大模型及其应用的安全性研究”选题, 构建了一个从红队样本、攻击脚本、行为监督机制到可视化演示系统的完整闭环。Argus Guardian 在工具调用真实执行前插入四层审计, 对智能体的工具调用、代码执行和文件访问进行实时判断。实验表明, 在当前 30 条离线样本上, 系统检出 22/22 条攻击, 阻断型误报为 0; 同时, Dashboard 支持 DeepSeek 在线红队、攻击链视图、自适应规则管理和一键演示验收, 能够较完整地展示大模型智能体应用中的攻击面与行为监督防御机制。

A 截图清单

正式提交报告时，建议补充以下图片：

1. `system_architecture.png`: 系统总体架构图；
2. `guardian_layers.png`: 四层 Guardian 判定流程图；
3. `dashboard_overview.png`: Dashboard 总览和四层统计截图；
4. `deepseek_attack_chain.png`: DeepSeek 批量矩阵与攻击链弹窗截图；
5. `rule_management.png`: 自适应规则管理页截图；
6. `benchmark_results.png`: Benchmark 结果截图；
7. `demo_acceptance.png`: 一键演示验收 SVG 快照或导出 PNG。

B 运行命令摘要

```
# 启动 Dashboard
.\.venv\Scripts\python scripts\dashboard_server.py --host 127.0.0.1 --port 8765

# 运行全部测试
.\.venv\Scripts\python -m pytest

# 生成离线评测结果
.\.venv\Scripts\python scripts\run_benchmark.py

# 检查前端 JS
node --check dashboard\app.js
```